MongoDB Aggregation Framework - Complete Guide

This document explains MongoDB Aggregation in detail including pipeline stages, operators, optimization, real-world use cases, and production best practices.

1. Introduction to Aggregation

Aggregation in MongoDB is used to process data records and return computed results. It is similar to SQL GROUP BY, COUNT, SUM, AVG operations but much more powerful. MongoDB uses an aggregation pipeline where documents pass through multiple stages and get transformed step-by-step.

Basic Example:

```
db.orders.aggregate([
  { $match: { status: "completed" } },
  { $group: { _id: "$customerId", total: { $sum: "$amount" } } }
])
```

2. Important Aggregation Stages

- $match – Filters documents (like WHERE in SQL)
- $group – Groups documents and performs calculations
- $project – Selects specific fields
- $sort – Sorts documents
- $limit – Limits number of documents
- $skip – Skips documents
- $lookup – Performs join between collections
- $unwind – Deconstructs arrays
- $facet – Multiple pipelines in one stage

3. Common Aggregation Operators

- $sum – Adds numeric values
- $avg – Calculates average
- $min / $max – Minimum & Maximum
- $count – Counts documents
- $push – Push values into array
- $addToSet – Unique values in array
- $first / $last – First or last document value

4. Advanced Real-World Example

Scenario: E-commerce Dashboard showing monthly revenue, total users, and top products.

```
db.orders.aggregate([
  { $match: { status: "completed" } },
  { $group: {
      _id: { month: { $month: "$createdAt" } },
      revenue: { $sum: "$amount" },
      totalOrders: { $sum: 1 }
  }},
  { $sort: { "_id.month": 1 } }
])
```

5. Aggregation Performance Optimization

- Always use $match early in pipeline
- Use indexes on filtered fields
- Avoid unnecessary $project stages
- Use allowDiskUse for large data
- Use $limit for pagination
- Analyze with explain() method

6. System Design Considerations (1M Users)

For applications with 1 million users, aggregation queries must be optimized carefully. Use proper indexing strategy, sharding for large collections, caching aggregated results in Redis, and pre-computed analytics collections for dashboards.

Best Practice: Use background jobs (cron) to pre-calculate heavy reports instead of calculating in real-time.